DAY 13 : Functions in python

—> A function is a block of code that can be executed only when it is called....

—> A fuction start with def keyword and the line is called as defination line , where we can define a function name

—>And if we want to execute the program in the function,need to call with the function name define at “**def line**”

syntax

def fun_name(parameters):

    pass

fun_name(arguments)

Eg:

def add(a,b):
print(a+b)
add(5,6)

Arguments

Positional Arguments

—> The arguments should be same at def line and calling, incase if they are not same number will raise an error

Eg:

def fibonacci():
num=0
num_2=1
print(num,num_2,end=' ')

```
for i in range(1,10):
    num_3=num+num_2
    num=num_2
    num_2=num_3
    print(num_3,end=' ')
```

fibonacci()

Default arguments

—> The default arguments where the function will only consider the data at calling, even though data present at def line

Eg:

def feb(a,b):
print(a+b)
feb([1,3],[5,6])

Keyword arguments

→Keyword arguments are sending in a pair(a=2), and the passing order is not consider...

Eg:

```
def data(age,name,batch,location):
print(name)
print(age)
print(batch)
print(location)
data(name='praveen',age=20,location='vizag',batch=6)
```

Variable length argument

→ Adding a(* call it as args) before a variable at parameters we can pass tuple of arguments and can be access with indexing

eg:

```
def all(*name):
print(name[4])
all('praveen','jayakumar','sanjay','sai','sony')
```

Keyword length argument

→adding a(** call it as args) before the parameter we can pass dic of arguments and can be access with dic methods..

eg:

```
def details(**data_):
print(data_)

details(name='praveen',age=20,location='vizag',batch=6)
```

---

return

→the **"return"** keyword used inside the function, once the return is executed means it will get back to calling with return values

eg:

```
def all(a,b):
return a-b
all(7,9)
```